MODULE 08 · DOMAIN 4 - TERRAFORM CONFIGURATION

Variables, Outputs, Locals, and Precedence

Three ways to name a value, and the five-level order that decides which one wins.

DOMAIN 4 of 8 COURSE WEIGHT 24% MODULE 8 of 27

admin@sthithapragna.com

Variables are a module's **inputs**. Outputs are its **return values**. Locals are its **working notes**.

The variable block

- **variable**, then a name. The name is how you reference it: var, a dot, then the name.
- **default** makes it optional. With no default, the variable is **required**.
- **type** constrains what may be assigned. **description** documents it for whoever uses the module.
- **sensitive** suppresses the value in Terraform's output. **nullable = false** forbids null being passed in.
- **validation** blocks add custom rules. Module twelve covers those in full.

> ON THE EXAM
>
> Declaring a variable and assigning it are separate acts. The block declares; the value arrives from one of five places.

Type constraints, briefly

- **Primitives:** string, number, bool. These cover most variables you will ever write.
- **Collections:** list, set and map, each holding one element type.
- **Structural:** object and tuple, where each attribute or position has its own type. Module nine.
- **any** accepts anything, and is a decision to do no checking rather than a type.
- Omit **type** entirely and Terraform accepts any value, which is the same as writing any.

> ON THE EXAM
>
> Terraform converts between primitives where it safely can, so the string "5" satisfies a number constraint.

Required variables, and automation

- If a required variable has no value, **Terraform prompts you for it interactively**.
- In a pipeline there is nobody to answer, so the run **hangs** until it times out.
- **-input=false** turns the prompt into an error, which fails fast and tells you which variable is missing.
- That is why module five listed it as an automation flag. This is the failure it prevents.
- A **default** is not always the fix. Some values genuinely should have no safe default.

> ON THE EXAM
>
> A hanging pipeline and a missing variable look nothing alike until you know Terraform prompts. -input=false makes them look the same.

Variable precedence, lowest to highest

SOURCE	NOTES
The default in the variable block	The lowest of all. A fallback, nothing more
Environment variables, TF_VAR_name	Higher than a default, and lower than every file
terraform.tfvars	Auto-loaded if present
terraform.tfvars.json	Auto-loaded, and beats the non-JSON file
*.auto.tfvars and .auto.tfvars.json	Auto-loaded, in lexical order of filename
-var and -var-file on the command line	Highest. In the order provided, so the last wins

ON THE EXAM

Later sources override earlier ones. HCP Terraform workspace variables sit at the top level, alongside the command line.

Three traps in that order

- **TF_VAR_ does not override a tfvars file.** It is second weakest. People assume the opposite constantly.
- **auto.tfvars files load in lexical order**, so a file starting with z overrides one starting with a.
- **A leftover .auto.tfvars from testing silently overrides your real values**, and nothing warns you.
- On the command line, **the last option wins**, whether it is -var or -var-file.
- So **-var-file=base.tfvars -var-file=prod.tfvars** layers production over base, which is the intended pattern.

> ON THE EXAM
>
> Auto-loaded means loaded whether you wanted it or not. That is convenient until a stray file is sitting in the directory.

Working an example

REGION IS SET IN	VALUE USED
default only	The default. Nothing else is in play
default, and TF_VAR_region	The environment variable. It beats the default
TF_VAR_region, and terraform.tfvars	terraform.tfvars. Files beat environment variables
terraform.tfvars, and b.auto.tfvars	b.auto.tfvars. Auto-loaded files rank higher
a.auto.tfvars, and b.auto.tfvars	b.auto.tfvars. Lexical order breaks the tie
b.auto.tfvars, and -var on the CLI	The command line. It beats every file

ON THE EXAM

Work upwards through the order and stop at the highest source present. There is never any merging or averaging.

Which files load themselves

- **terraform.tfvars** and **terraform.tfvars.json** are loaded automatically if present.
- Any file ending **.auto.tfvars** or **.auto.tfvars.json** is loaded automatically.
- **Any other name is not.** A file called prod.tfvars is ignored unless you pass -var-file.
- That is deliberate, and it is how per-environment variable files are meant to work.
- **tfvars files are ordinary text.** Committing secrets to one is module one's mistake in a new file.

> ON THE EXAM
>
> Naming a file prod.tfvars and expecting it to load is a common error. Only terraform.tfvars and *.auto.tfvars load themselves.

Where the precedence order does not apply

- **That order is for root module variables only.** Child modules do not participate in it.
- A child module's variables are set **solely by the arguments in its module block**.
- **-var and -var-file cannot reach a child module.** Neither can TF_VAR_ environment variables.
- To make a child module's input configurable, the root module declares its own variable and passes it through.
- That pass-through is the normal pattern, and it is why root modules often look like a list of variables.

> ON THE EXAM
>
> A module block argument is not 'like' setting a variable. It is the only way a child module variable ever gets a value.

The output block

- **output**, then a name, then a **value** expression. That is the minimum.
- In a **child module**, outputs are the only way a caller can read anything from inside.
- In the **root module**, outputs are printed after apply and read with terraform output.
- **sensitive** redacts the value in output. **description** documents it. **depends_on** is available but rarely needed.
- Outputs are stored in **state**, which is why terraform output works without re-planning.

> ON THE EXAM
>
> A module's outputs are its API. Anything not declared as an output is unreachable from outside, by design.

Local values

- Declared in a **locals** block, note the plural, and referenced as local, a dot, then the name.
- **Nobody can set a local from outside.** It is computed inside the module, from variables, resources or literals.
- Use one when the same expression appears several times, so a change happens in one place.
- Also use one to give a complicated expression a **readable name**, which is often the bigger win.
- **Overuse hides things.** A local defined far from where it is used is harder to follow, not easier.

> ON THE EXAM
>
> The block is 'locals' plural; the reference is 'local' singular. That inconsistency catches people writing configuration from memory.

Variable or local?

A VARIABLE WHEN

Somebody else should decide

- The value differs per environment or caller
- It belongs in the module's interface
- It should be documented and typed
- Settable from five different places

A LOCAL WHEN

The module decides

- Computed from other values
- The same in every use of the module
- An implementation detail, not an interface
- Settable from nowhere at all

THE TIEBREAKER

A variable with a default that nobody ever overrides is usually a local wearing the wrong block type.

The three, side by side

	SET FROM OUTSIDE? READ FROM OUTSIDE?
variable	Set from outside: yes. Read from outside: no
output	Set from outside: no. Read from outside: yes
local	Set from outside: no. Read from outside: no

ON THE EXAM

Variables flow in, outputs flow out, locals stay put. That single table answers a surprising number of questions.

sensitive, one more time

- **sensitive = true** is available on both variables and outputs. It redacts the value in Terraform's output.
- Terraform also **tracks it through expressions**, so a value derived from a sensitive one is redacted too.
- **It does nothing about state,** where the value sits in plain text. Module four.
- **It does nothing about plan files** either, where it is stored in cleartext. Module five.
- **terraform output -raw** and **-json** reveal sensitive outputs deliberately, so scripts can use them.

> ON THE EXAM
>
> Third module in a row making the same point. sensitive controls display. Storage is a separate problem, solved in module thirteen.

How files are usually organised

- Terraform reads **every .tf file in the directory** and treats them as one configuration. Filenames carry no meaning.
- Convention, not requirement: **variables.tf**, **outputs.tf**, **main.tf**, **versions.tf**.
- **Subdirectories are not read.** A directory is a module, and its children are separate modules.
- So splitting files changes nothing functionally. It is entirely for humans.
- The one file that is special is **terraform.tfvars**, and the .auto.tfvars pattern.

> ON THE EXAM
>
> Since filenames carry no meaning, a question about which file a block 'must' go in is almost always testing that fact.

Reading the scenario

THE WORDING SAYS	IT IS POINTING AT
An exported TF_VAR_ value is being ignored	A tfvars file is overriding it. Env vars are nearly lowest
A value changed and nobody edited anything	A stray .auto.tfvars, loaded automatically
We need a value from inside a child module	The module must declare an output. There is no other route
The pipeline hangs with no error	A required variable, and a missing -input=false
Every caller passes the same value	It should probably be a local, not a variable
Two -var-file options were passed	The last one wins, which is the layering pattern

ON THE EXAM

Precedence questions are mechanical. Recall the order, find the highest source in play, and that value wins.

A pipeline sets TF_VAR_environment to staging before running terraform apply against the staging account. The repository contains a terraform.tfvars file, committed a year ago, which sets environment = "production".

Engineers have exported the variable correctly, confirmed it with echo, and the deployment still behaves as production.

Why is the environment variable being ignored?

A TF_VAR_ variables must be exported in the same shell that runs Terraform, which the pipeline does not do

B terraform.tfvars has higher precedence than environment variables and overrides it

C The variable name must match exactly including case, and TF_VAR_ names are case-sensitive

D Environment variables are only read when no variable block declares a default

◆ PAUSE AND DECIDE

Why is the environment variable being ignored?

A Wrong shell

A real class of bug, and ruled out by the scenario, which says the export was confirmed. Read that sentence as the question closing a door.

B tfvars outranks env vars

Correct. Environment variables sit second from the bottom, above only the default. Every tfvars file beats them.

C Case sensitivity

Names are case-sensitive, and a mismatch would mean the variable was simply unset, producing a prompt or an error rather than a wrong value.

D Only read without a default

Invents a rule. Environment variables are always consulted; they just lose to anything above them in the order.

THE SIGNAL

The instinct that environment variables override files comes from other tools. In Terraform they are nearly the weakest source.

> A root configuration calls a networking module that creates a VPC, subnets and a NAT gateway. A new requirement needs the NAT gateway's public IP address, to add it to a partner's allow list.
>
> An engineer writes a reference to the NAT gateway resource inside the module, using module dot network followed by the resource address. Terraform reports that the reference is invalid.

What is required?

A Use terraform_remote_state to read the module's internal resources

B Add an output to the networking module exposing the IP, then reference module.network and that output name

C Move the NAT gateway into the root module, since module internals cannot be referenced

D Reference it through the state file path instead, using terraform state show

◆ PAUSE AND DECIDE

What is required?

A Use remote state

Remote state reads another configuration's root outputs. This module is in the same configuration, and outputs are still the only interface.

B Declare an output

Correct. A module's outputs are its entire public interface, so exposing a value means the module author declares it.

C Move it to root

Would work and dismantles the module to read one value. Modules exist precisely so internals can stay internal.

D Read it via state show

An inspection command, not an expression. It prints for a human and cannot supply a value to configuration.

THE SIGNAL

Encapsulation is not a side effect here. A module exposes exactly what its author declared, and nothing else is reachable.

> A module declares a variable called name_prefix with a default built from two other variables, joined with a hyphen. It is documented, typed, and has never been overridden by any of the eleven teams using the module.
>
> A reviewer suggests the declaration is the wrong shape.

What should it be instead, and why?

A. Leave it. A variable with a default is the correct way to provide a computed convenience value

B. A local value, since it is computed inside the module and nobody should be setting it

C. An output, so consuming configurations can see the prefix that was used

D. A variable with nullable = false, which prevents callers from overriding it

◆ PAUSE AND DECIDE

What should it be instead, and why?

A Leave it as a variable

A variable default cannot reference other variables at all, so the described declaration would not even be valid. It is also an interface nobody asked for.

B Make it a local

Correct. It is computed inside the module from other values, and no caller should decide it. That is exactly what a local is for.

C Make it an output

An output would let callers read it, which nobody has asked for, and would not stop it being settable. It answers a different question.

D nullable = false

Rejects null being passed. It does not stop a caller passing some other value, so the unwanted knob is still there.

THE SIGNAL

Every variable is a promise to support that input. Eleven teams and zero overrides means the promise was never needed.

> A newly added variable called alert_email has no default, because there is no sensible one. It is supplied through a tfvars file in every environment except one, where somebody forgot.
>
> In that environment the nightly pipeline job now runs for two hours and is killed by the job timeout. The log ends after the provider initialisation lines, with no error.

What is happening, and what is the immediate fix?

A. The provider is rate limiting. Add -parallelism to reduce concurrent requests

B. Terraform is waiting at an interactive prompt for the missing variable. Add -input=false

C. The state lock is held by another run. Add -lock-timeout

D. The variable must be given a default, since Terraform cannot run without one

◆ PAUSE AND DECIDE

What is happening, and what is the immediate fix?

A Rate limiting

Would produce retries and eventually errors in the log. The log simply stops, which is the shape of something waiting, not something failing.

B Waiting at a prompt

Correct. A required variable with no value makes Terraform prompt. -input=false turns that into an immediate, named error.

C Lock contention

Terraform reports lock contention explicitly and says who holds the lock. Silence is the tell that this is not it.

D Must have a default

False, and undesirable. Variables without defaults are normal and correct where no safe default exists. A default here would silently send alerts nowhere.

THE SIGNAL

No error message is itself the diagnostic. Terraform is loud about failures and silent only when it is waiting for a person.

A production configuration has always set instance_count = 6 in terraform.tfvars. During a load test three weeks ago, an engineer created a file named zz-loadtest.auto.tfvars setting instance_count = 1, and never removed it.

Since then production has been running one instance. Nobody changed terraform.tfvars, and every plan has reported no changes.

Why did this happen without warning?

A Files ending .auto.tfvars are loaded automatically and override terraform.tfvars

B The file name begins with z, so it is loaded last and wins over all other sources

C Terraform merges conflicting values and chooses the lower one for numeric variables

D The plan reported no changes because state was never refreshed after the load test

◆ PAUSE AND DECIDE

Why did this happen without warning?

A auto.tfvars is auto-loaded and outranks

Correct. Auto-loaded means loaded whether or not you intended it, and .auto.tfvars sits above terraform.tfvars in the order.

B The z makes it win

Lexical order only breaks ties between several .auto.tfvars files. It does not lift the file above the command line, so 'all other sources' is wrong.

C Terraform merges values

Invents a rule. Terraform never merges scalar values or picks between them; the highest-precedence source simply wins.

D State was not refreshed

The plan is correct. One instance is genuinely desired, given the values in force. Nothing is drifting.

THE SIGNAL

Every plan was honest. It compared configuration against state, and the configuration really did ask for one instance.

Module 8 recap

- **Variables in, outputs out, locals stay put.** That table answers most design questions.
- **TF_VAR_ is second weakest,** above only the default. Every tfvars file beats it.
- **terraform.tfvars and *.auto.tfvars load themselves.** Anything else needs -var-file.
- **Command line is strongest, last option wins.** Which is why -var-file layering works.
- **A module's outputs are its only interface.** Undeclared internals are unreachable.
- **A required variable makes Terraform prompt.** In a pipeline that looks like a hang. Use -input=false.

> ON THE EXAM
>
> Variable defaults must be literal. They cannot reference other variables, which is often the hint that you wanted a local.